CHAPITRE 1 – Problématique des systèmes de gestion de fichiers

Par Robert DIASSÉ

1. Introduction

Avant l'apparition des bases de données modernes, les organisations utilisaient des **systèmes de gestion de fichiers** pour stocker et organiser les informations.
Un système de fichiers consiste simplement à enregistrer les données dans des **fichiers textes, documents, tableurs**, souvent séparés et sans lien.

Ce chapitre montre :

- comment fonctionnent ces systèmes,
- leurs limites,
- et pourquoi les bases de données ont été inventées.

---

2. Qu'est-ce qu'un système de gestion de fichiers ?

Un **système de gestion de fichiers** est une manière simple de stocker les informations dans des fichiers indépendants, souvent enregistrés sur le disque.

Exemples :

- Un fichier Excel pour la liste des étudiants
- Un fichier Word pour les enseignants
- Un autre fichier Excel pour les notes
- Un fichier texte pour les inscriptions

Chaque fichier fonctionne **indépendamment**, sans connexion automatique aux autres.

Caractéristiques :

- Les données sont enregistrées sous forme de fichiers isolés.
- La gestion se fait par le système d'exploitation (Windows, Linux…).
- Les programmes accèdent directement aux fichiers via le code.

Exemple concret :

Supposons une université qui stocke l'information ainsi :

- Étudiants : `etudiants.xlsx`
- Cours : `cours.xlsx`
- Inscriptions : `inscriptions.xlsx`

Si un étudiant change son nom, il faut le modifier **dans chaque fichier où il apparaît**, manuellement.

---

3. Fonctionnement d'un système de gestion de fichiers

a) Stockage des données

Chaque application crée et gère ses propres fichiers.

Exemple :
Le logiciel « Gestion des notes » gère son fichier `notes.txt`.
Le logiciel « Gestion des inscriptions » gère `inscriptions.txt`.
Aucun lien direct n'existe entre les deux.

b) Accès aux données

Le code du programme contient les instructions :

- Pour trouver le fichier
- Pour lire les données
- Pour les modifier
- Pour les réécrire dans le fichier

c) Problème majeur

Chaque programme doit :

- connaître le format du fichier,
- savoir comment les données sont organisées,
- gérer manuellement les erreurs.

Cela crée un système fragile, dépendant du code, difficile à maintenir.

---

4. Limites du système de gestion de fichiers

Voici les limites les plus importantes, que les SGBD ont été inventés pour résoudre.

---

1. Redondance des données (répétitions inutiles)

Les mêmes informations se retrouvent dans plusieurs fichiers.

Exemple :
Le nom d'un étudiant apparaît dans :

- le fichier des inscriptions
- le fichier des notes
- le fichier des paiements

Problème :

- Occupation inutile de l'espace
- Risques d'incohérence si une modification est oubliée dans un fichier

---

2. Incohérence des données

Lorsqu'on modifie un fichier mais pas l'autre, les données deviennent contradictoires.

Exemple :

- Dans `etudiants.xlsx` → "Moussa Sow"
- Dans `inscriptions.xlsx` → "Moussa Soh"

Le système est incapable de savoir quelle version est correcte.

---

3. Difficulté d'accès et de recherche

Pour faire une recherche, il faut souvent :

- ouvrir plusieurs fichiers
- écrire du code spécifique
- parcourir manuellement toutes les données

Pas de langage de requête comme SQL.

---

4. Manque de sécurité

Les fichiers sont accessibles :

- sans contrôle précis,
- sans droits d'accès,
- sans confidentialité des données.

Impossible de dire :

- « tel utilisateur peut modifier mais pas supprimer »
- « tel utilisateur ne voit que les données de son département »

---

5. Pas de gestion de la concurrence

Dans une université :

- deux secrétaires modifient un fichier en même temps,
- l'un enregistre,
- l'autre écrase tout sans le savoir.

Aucun mécanisme de verrouillage ou de versionnement.

---

6. Pas d'intégrité des données

Impossible d'imposer des règles comme :

- « chaque étudiant doit avoir un identifiant unique »
- « un paiement doit être lié à un étudiant existant »
- « une inscription doit référencer un cours existant »

Tout doit être vérifié manuellement.

---

7. Difficulté d'évolution

Si on veut :

- ajouter une nouvelle colonne,
- changer le format du fichier,
- fusionner des données,

… alors il faut reprogrammer les applications, parfois entièrement.

---

5. Pourquoi les bases de données ont été inventées ?

Les bases de données répondent à **toutes les limites** évoquées.

Elles apportent :

1. Moins de redondance

Les données sont stockées **une seule fois** et reliées grâce aux clés.

2. Cohérence garantie

Un SGBD maintient automatiquement l'intégrité grâce aux contraintes :

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- CHECK

3. Sécurité et droits d’accès

On peut décider :

- qui peut lire,
- qui peut écrire,
- qui peut supprimer.

4. Accès rapide et puissant

Grâce au langage SQL :

- recherche,
- filtrage,
- tri,
- regroupement,
- jointure…

5. Concurrence maîtrisée

Plusieurs utilisateurs peuvent :

- consulter,
- modifier,
- insérer,

en même temps, sans destruction de données.

6. Sauvegardes et récupération

Un SGBD inclut :

- des sauvegardes automatiques,
- la restauration après panne.

7. Souplesse d’évolution

On peut :

- ajouter une table,
- modifier une structure,
- changer un type de données,

sans réécrire les programmes existants.

---

6. Conclusion du chapitre

Les SGBD ont été créés pour résoudre tous les problèmes des systèmes de gestion de fichiers traditionnels.
Ils permettent :

- de structurer les données,
- d’éviter les anomalies,
- de garantir la sécurité,
- de faciliter les recherches,
- et d'assurer la cohérence.

Ce chapitre sert de base pour comprendre **pourquoi** la notion de base de données est fondamentale avant d'apprendre à les concevoir.